

Project Confirmation

Real-Time Transit System

INFO 4290 – Integration Project II (S50)

Chloe Chang, James Sinclair, Sanjeet Khangura, Kumardeep Singh

May 18, 2026

Project Title: Real-Time Transit System

Team Members: Chloe Chang (100441028), James Sinclair (100443197), Sanjeet Khangura (100379050), Kumardeep Singh (100369778)

Project Summary

This project addresses the problem of unreliable transit information by using real-time and historical bus data to help users better understand delays, route reliability, and overall bus performance. While current transit apps mainly show scheduled times and basic live tracking, they often do not explain delays, bus bunching, or routes that regularly run behind. Our system collects and analyzes transit data using machine learning to give users predictions and insights into route performance.

The application should provide frequent transit commuters, transit enthusiasts, and route planners with more detailed information and analysis about their transit network. The system will provide users with detailed historical data, including route reliability and schedule adherence. It will also provide real-time analysis of the current state of the system, identifying bus bunching and route conditions. Finally, it will calculate and provide users with schedule predictions based on historical data and current route conditions.

What makes the project unique is that it displays scheduled, real-time, and predicted route times along with detailed alerts to give users a better understanding of how bus routes perform.

Technology Stack

Below are the various technologies we have selected to complete our project.

Front End	
JavaScript Library	React (using NextJS)
Back End	
API Server	Go (using Gin)
Ingest Service	Python (using Pandas)
ML Training Service	Python (using Pandas and Scikit-learn)
Database	PostgreSQL (using TimescaleDB)
Infrastructure	
Build Pipeline	GitHub Actions
Container Management	Kubernetes
Hosting Platform	Local Server (fall back to Google Cloud)

GitHub Repository Link

<https://github.com/SanjeetKhangura/RealTimeTransit>

Project Scope

Data Ingest & Storage

The system queries the transit agency's GTFS API and stores the current state of the transit network. The system maintains a repository of time-series transit network data. This data is used to calculate a baseline reliability profile of each route and to generate a prediction model based on historical conditions.

Data Processing & Analysis

The system performs analysis of current route conditions, calculating variance from typical route conditions, detecting bus bunching and missing buses, and recording schedule adherence.

Machine Learning & Prediction

The system uses historical data to generate a machine learning model of bus arrival times. The model is then used to estimate the likely arrival times of buses based on current route conditions. The system continues to train and improve the model as transit data is collected.

User Interface / Web Dashboard

The system presents the data it has collected and the calculations it has performed on a web dashboard. This dashboard allows users to view the current conditions, estimates of future arrival times, and historical insights.

Modularity & Resilience

The system is modular and resilient. Different functions of the system are performed by independent modules. These modules are deployed in separate containers (Docker/Kubernetes). A failure of one module does not impact the function of other modules. For example, if the machine learning module crashes, the data ingest and API modules remain functional. In addition, should one or more of the modules become unavailable on one host machine, the container orchestrator could be configured to start the services on another machine or cloud platform.

Risks and Concerns

Limited Training Data

We may not be able to collect enough historical transit data to train the machine learning model effectively, which could reduce prediction accuracy.

System Complexity

The project includes multiple interconnected modules, making implementation and integration more complex. These modules include the data ingest module, the API module, the machine learning module, the user interface, and other supporting infrastructure like the database. More details are shown in *Figure 4* and *Figure 5*.

Machine Learning Accuracy

The prediction model may produce inaccurate results if the data is inconsistent, incomplete, or affected by unexpected transit conditions.

Database Storage Constraints

Continuously collecting and storing real-time and historical transit data could require significant database space over time.

Figures

User Interface Design

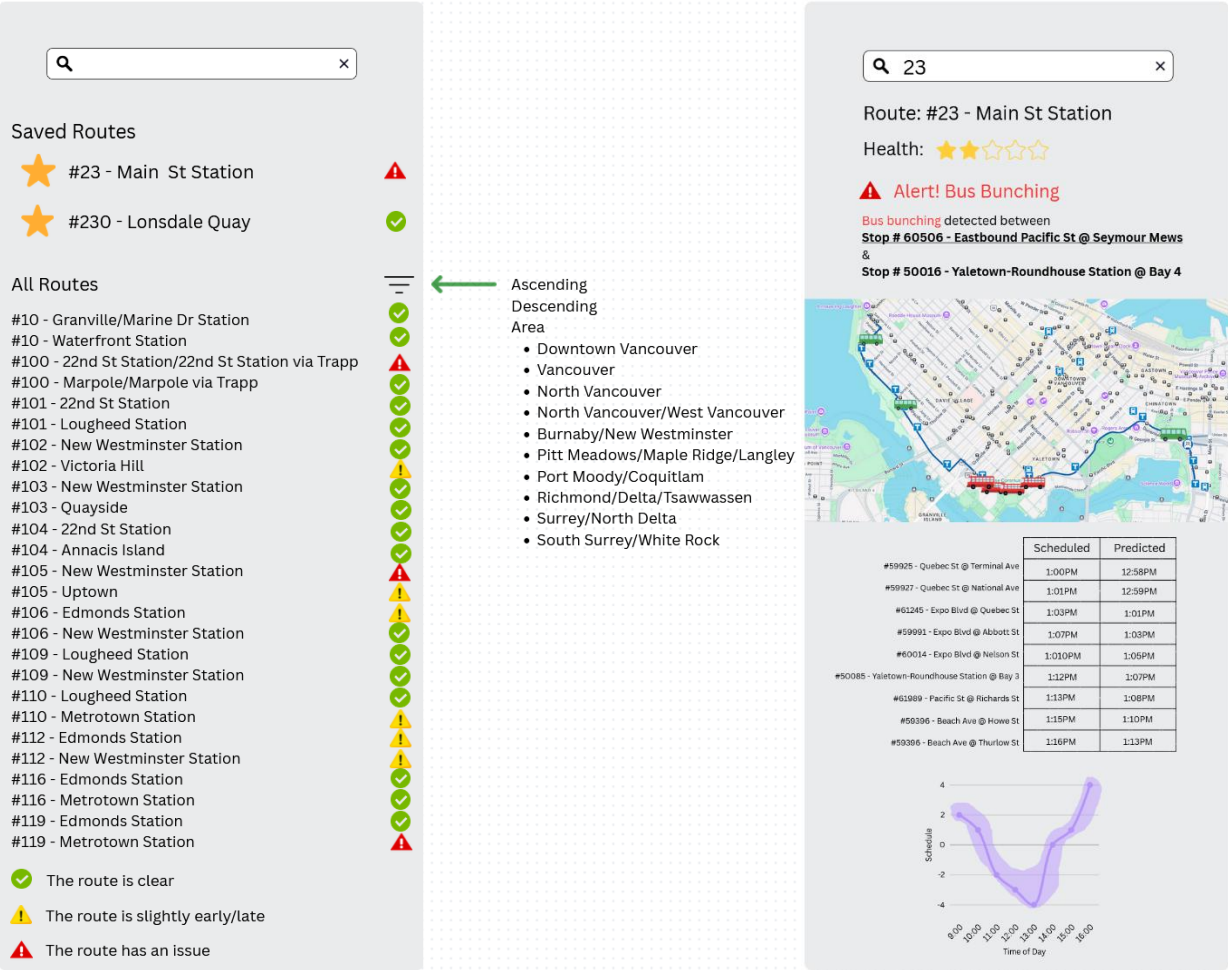


Figure 1

Figure 2

Route List

Figure 1 is the main page of the web application, and the first screen users see. It provides an overview of all available bus routes and helps users quickly find the ones they need.

At the top of the page, there is a search bar where users can enter a bus number to find a specific route. This makes it easy to go directly to a route without scrolling through the full list.

Below the search bar, all routes are displayed in a list. Users can filter this list by route number or by area, which helps them focus on buses that are relevant to them or nearby. Each route includes an icon that shows its current status, such as running normally,

slightly delayed, or experiencing a larger issue. This allows users to quickly understand how a route is performing at a glance.

The page also displays any routes the user has saved, making them easy to access without needing to search again.

Route Display

Figure 2 is displayed when a user selects a specific bus route, either by searching or choosing one from the main list. This page provides a detailed overview of the selected route and its current performance.

At the top of the page, the name of the route is shown along with an overall health rating. This rating is based on how well the buses on the route are performing. For example, a 5-star rating indicates that buses are running on time, while a lower rating indicates delays or issues like bus bunching.

If there is a problem on the route, an alert is displayed describing the issue and where it is occurring. For instance, the system may indicate bus bunching between two specific stops, helping users quickly understand what is happening and where.

The page also includes a real-time map that shows the route and the current positions of buses. Buses are coloured (red or green) based on their status, making it easy to spot delays or clustering along the route.

In the second half of the page, users can view both the scheduled arrival times and predicted arrival times for each stop. The predicted times are generated using machine learning based on historical data, providing a more accurate estimate of when the bus will arrive.

Finally, a graph is included to show trends in bus performance over time. It displays how early or late buses tend to be at different times of the day, giving users additional insight into patterns on the route.

System Design

Flowchart

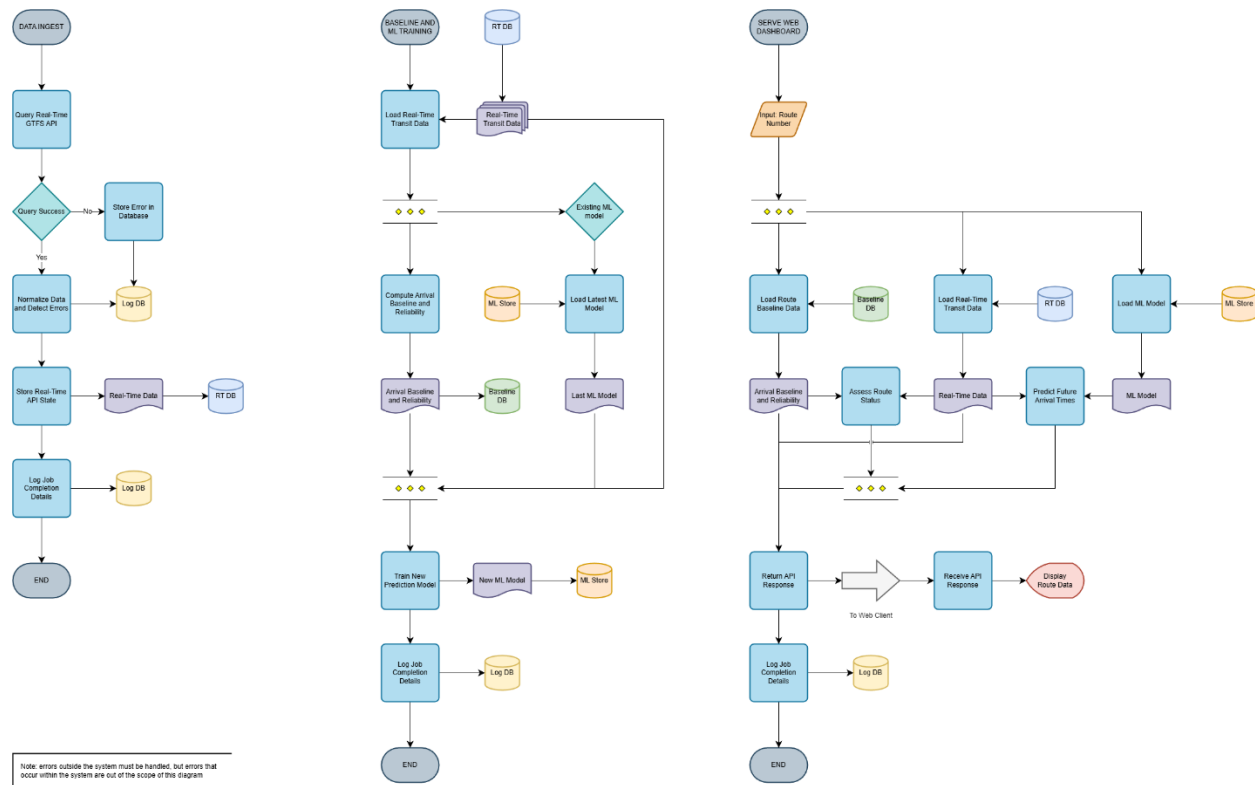


Figure 3

Figure 3 shows the flow of actions and data for the three core functions of the system. This illustrates the processes of the data ingest, machine learning, and web dashboard display processes. These flows do not distinguish between tasks performed by different modules.

Component Diagram

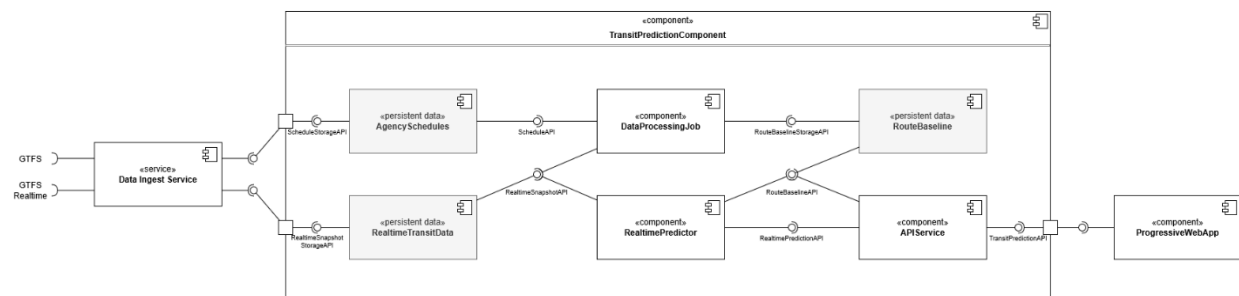


Figure 4

Figure 4 shows the relationship between the different modules within the system. This includes the various components that perform different tasks as well as how they connect

to various types of persistent data. This data is stored in a time-series database, a document database, as well as an object storage service.

Deployment Diagram

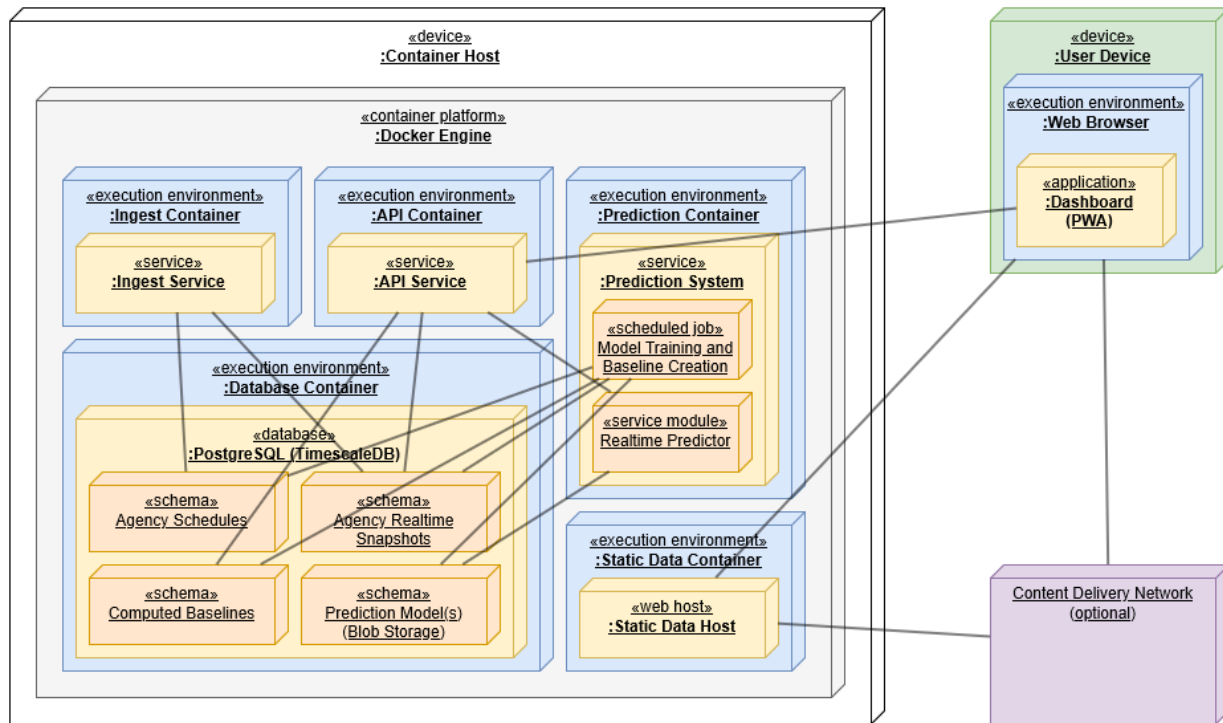


Figure 5

Figure 5 illustrates the deployment and relationships of the various modules in the system. This includes containers and services managed by the container orchestrator, as well as the web dashboard served to the user.